שידור מידע בין 2 מחשבים באמצעות מצלמה ו-QR Codes

סדנה בתקשורת מחשבים
20588, 2021ג'
דיאנה כהן

רקע

חשיבות היכולת לתקשר ללא חיבור לאינטרנט, וללא WIFI - למשל במהלך טיסה, או באזורים ללא קליטה.

ניצול משאבים קיימים ונגישים כגון מצלמות, בעיקר בעידן טלפונים חכמים.

חשיבות לימודית - הבנה מעמיקה של פעולת התקשורת ומימוש של פרוטוקול.

דרישות הפרויקט

בהינתן 2 מחשבים שלכל אחד מהם מצלמה, כאשר המצלמה של כל אחד מהם מכוונת למסך של המחשב השני.

בפרויקט זה, יעשה ניסיון לשדר קובץ בין 2 המחשבים ע"י הקרנת תוכנו בחלקים באמצעות QR Codes. כל QR Code יכול לייצג כ-40 תווים (תלוי בגרסה).

ברגע שהמצלמה של מחשב אחד תזהה QR Code שמוקרן על המסך ממול, היא תפענח את תוכנו וכך ישודר תוכנו של הקובץ בין המחשבים.

צריך יהיה לבצע calibration לשני המחשבים כך שהקרנת QR Code תיקלט היטב במצלמה של המחשב ממול.

תקציר

אפליקציית שרת-לקוח, בה הלקוח מבקש מהשרת להעביר בתקשורת את קובץ המקור שנמצא על השרת אל קובץ היעד שיווצר אצל הלקוח.

שירות מקושר עם אישורים (Acknowledged connection-oriented service).

מימוש המזכיר את פרוטוקול 3 לשכבת הערוץ (מקורס מבוא לרשתות תקשורת מחשבים).

מערכת הפעלה UBUNTU 20.04.

תכנות בשפת GO החדשנית.

שימוש בספריות צד שלישי לצורכי עבודה עם QR Codes והמצלמה.

ספריות צד שלישי

QR code encoder:

```
$ go get github.com/skip2/go-qrcode
```

Webcam capture frame into single image:

```
$ go get github.com/blackjack/webcam
```

QR code decoder:

```
$ go get github.com/liyue201/goqr
```

תכנון הפתרון

הקמת קשר:

- השרת עולה ושולח הודעת אתחול - למשל כתובת ה-IP שלו.
- הלקוח עולה, קורא את ההודעה של השרת, ובתגובה שולח הודעת אתחול משלו - נתיב הקובץ המבוקש מהשרת וגודל החוצץ של הלקוח.

ניהול הקשר - דומה לפרוטוקול 3:

- הודעות אלה מכילות מספור המנה (0 ו-1 לסירוגין) בבית הראשון ואחריו את תוכן המנה.
- השרת קורא את קובץ המקור לפי גודל החוצץ, מחולל QR Code המכיל מספור ותוכן ומציג אותו. אין שידור חוזר, כיוון שהקוד ימשיך להיות מוצג.
- הלקוח סורק ומפענח את ה-QR Code. אם מדובר במספר מנה שהלקוח מצפה לקבל, אזי הוא שולח ACK (גם הוא בעצם QRCODE שמוצג).
- השרת סורק ומפענח את ה-QR Code. אם מדובר במספר ACK תואם למנה שנשלחה, השרת מתקדם לקריאת המנה הבאה מהקובץ.
- אם במהלך הקשר קוראת שגיאה אצל אחד הצדדים, הוא שולח הודעת ERR לצד השני ומסיים את הקשר (ללא המתנה לתגובה).

סיום הקשר:

- אם במהלך ניהול הקשר מתקבלת הודעת ERR אצל אחד מהצדדים, הקשר מנותק מיידית, כיוון שלא יגיעו הודעות נוספות.
- אם שידור הקובץ מסתיים בהצלחה, אזי השרת שולח הודעת FIN לניתוק הקשר, ומסיים את פעולתו. הלקוח יסיים גם הוא את פעולתו, כאשר יקבל את הודעת FIN.

תזכורת - פרוטוקול 3 לשכבת הערוץ

```
/* Protocol 3 (PAR) allows unidirectional data flow over an unreliable channel. */

#define MAX_SEQ 1                          /* must be 1 for protocol 3 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void sender3(void)
{
  seq_nr next_frame_to_send;               /* seq number of next outgoing frame */
  frame s;                                 /* scratch variable */
  packet buffer;                           /* buffer for an outbound packet */
  event_type event;

  next_frame_to_send = 0;                  /* initialize outbound sequence numbers */
  from_network_layer(&buffer);             /* fetch first packet */
  while (true) {
      s.info = buffer;                     /* construct a frame for transmission */
      s.seq = next_frame_to_send;          /* insert sequence number in frame */
      to_physical_layer(&s);               /* send it on its way */
      start_timer(s.seq);                  /* if answer takes too long, time out */
      wait_for_event(&event);              /* frame_arrival, cksum_err, timeout */
      if (event == frame_arrival) {
          from_physical_layer(&s);         /* get the acknowledgement */
          if (s.ack == next_frame_to_send) {
              stop_timer(s.ack);           /* turn the timer off */
              from_network_layer(&buffer); /* get the next one to send */
              inc(next_frame_to_send);     /* invert next_frame_to_send */
          }
      }
  }
}
```

```
void receiver3(void)
{
  seq_nr frame_expected;
  frame r, s;
  event_type event;

  frame_expected = 0;
  while (true) {
      wait_for_event(&event);              /* possibilities: frame_arrival, cksum_err */
      if (event == frame_arrival) {        /* a valid frame has arrived */
          from_physical_layer(&r);         /* go get the newly arrived frame */
          if (r.seq == frame_expected) {   /* this is what we have been waiting for */
              to_network_layer(&r.info);   /* pass the data to the network layer */
              inc(frame_expected);         /* next time expect the other sequence nr */
          }
          s.ack = 1 – frame_expected;      /* tell which frame is being acked */
          to_physical_layer(&s);           /* send acknowledgement */
      }
  }
}
```

Figure 3-14. A positive acknowledgement with retransmission protocol.

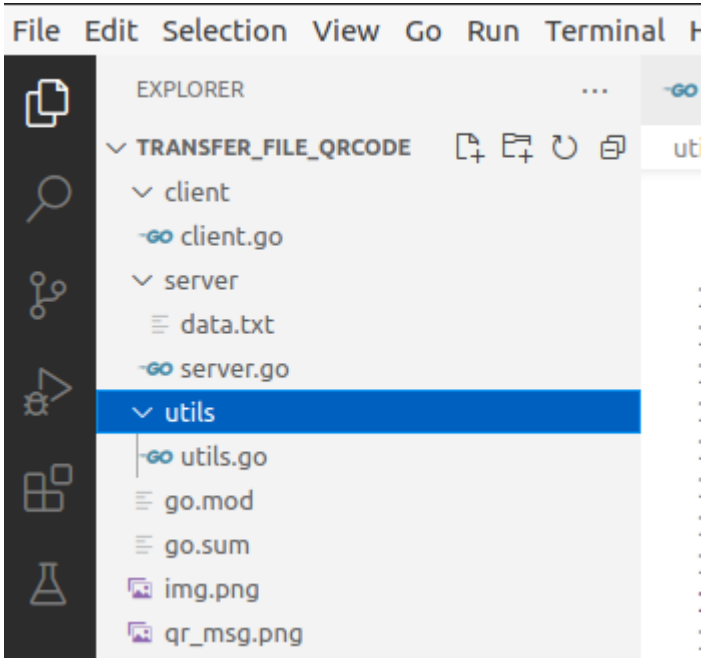
שלבי מימוש

- שלב ראשון - תכנון התשתית עם תקשורת TCP בין הצדדים להעברת הקובץ.
- שלב שני - בנוסף לתקשורת TCP דו כיוונית, הצגת ההודעה הנשלחת כ-QRCODE.
- שלב שלישי - ניתוק ראשוני של ה-TCP:
  - השרת שולח הודעה כ-QRCODE ומקבל ב-TCP.
  - הלקוח סורק עם המצלמה את ה-QR Code (מקבל הודעה) ושולח ACK ב-TCP.
- שלב סופי - ניתוק סופי של ה-TCP ומעבר לתווך דו כיווני של המצלמות.

תכנון בדיקות ודגשים במימוש

- טיפול בשגיאות בכל מקום אפשרי.
- הוספת הודעות בקרה עבור כל שלבי הריצה של התוכנית.
- שליטה בהודעות בקרה ע"י קבוע הבקרה DEBUG.
- הודעות בקרה בכניסה ויציאה מהפונקציות, בדומה ל-TRACE.
- בדיקות תקינות עבודה מול תיקיות צד שלישי:
  - כתיבת QR Code לתוך PNG ופענוח ממנו.
  - שמירת FRAME בודד מהמצלמה לתוך PNG.
  - לאחר סיום תקין של התווך, ניתוק עבודה מול קבצים.
- הרצות שונות וכוונון של קבועים כגון זמן השהייה, כמות ניסיונות חוזרים וכו'.
- זמן השהייה הולך וגדל כתלות של קבוע השהייה ומספר הניסיון, ובכך מונע תקיעה הדדית בגלל תזמון רע.

מבנה הפרויקט



תיקיית UTILS ובה UTILS.GO המכיל מימושים עיקריים של התשתית ופונקציות לשימוש חוזר עבור השרת והלקוח.

תיקיית CLIENT ובה CLIENT.GO להרצת צד הלקוח. זו היא גם תיקיית היעד עבור הקובץ.

תיקיית SERVER ובה SERVER.GO להרצת צד השרת. זו היא תיקיית המקור עבור קבצים הזמינים להעברה.

קבצי PNG בתיקיית הפרויקט הם תוצר הבדיקות מול ספריות צד שלישי. לא נחוצים לריצת הפרויקט.

הרצת הפרויקט מתוך ספריית הפרויקט

הרצת צד שרת - פרמטרים:

1. מצלמה של השרת

```
$ go run server/server.go /dev/video2
```

הרצת צד לקוח - פרמטרים:

2. קובץ מקור להורדה מהשרת
3. קובץ יעד בצד הלקוח
4. גודל החוצץ של הלקוח - גודל ההודעה במספר הבתים שתשודר מהשרת ללקוח
5. מצלמה של הלקוח

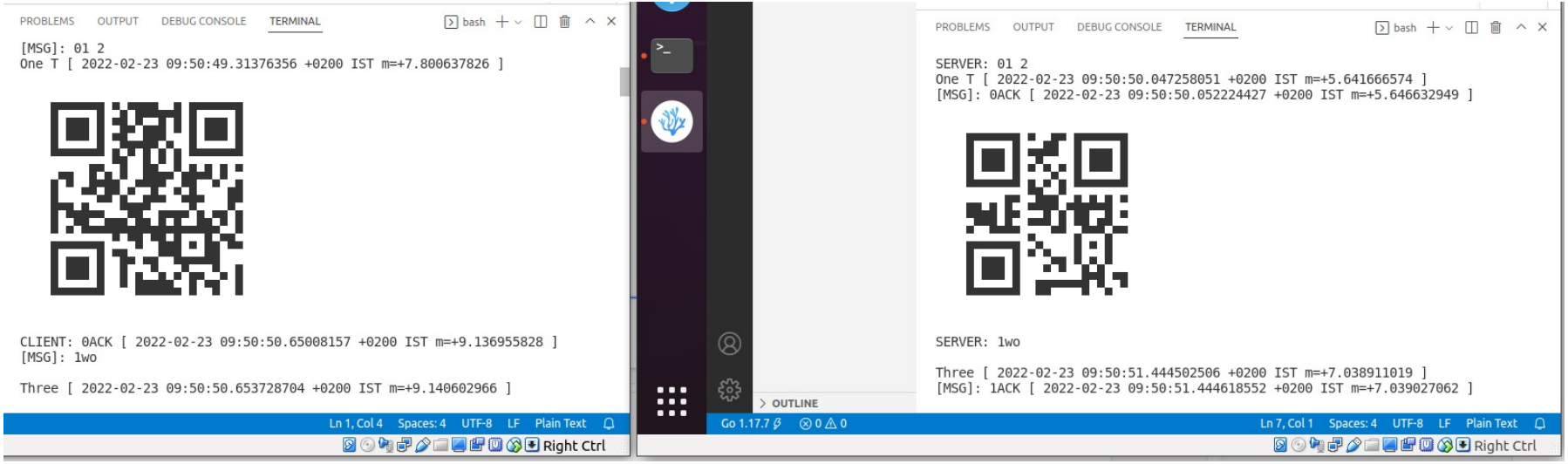
```
$ go run client/client.go server/data.txt client/client_copy.txt 10 /dev/video0
```

הדגמה בתמונות - הקמת הקשר ע"י החלפת הודעות אתחול

הרצת צד שרת ומיד אחריו הרצת צד לקוח. החלפת הודעות אתחול לצורך הקמת הקשר.

הדגמה בתמונות - ניהול הקשר ע"י שידור הודעה וקבלת ACK

לצורכי הדגמה ובקרה - הדפסה למסך של הודעה יוצאת [MSG] המוקרנת כ-QR Code, הדפסה של הודעה נכנסת מהצד השני (ה-QR Code שפוענח מהמצלמה).



הדגמה בתמונות - סיום הקשר

העברת הקובץ הושלמה בהצלחה - השרת סיים לקרוא קובץ, שולח הודעת FIN ובכך מסיים את הקשר מול הלקוח ומסיים את ריצתו. הלקוח מקבל הודעה זו ומסיים גם כן.

הדגמה על 2 מחשבים שונים

הדגמה מתוך סביבת פיתוח על מחשב אחד המריץ 2 תוכניות